

## API REST — Backend CRUD

### Entrega 2 — Sistema de Gestão para Concessionária de Veículos

Esta documentação descreve os contratos REST implementados no backend da Entrega 2 para as tabelas principais do banco oficial: **CLIENTES**, **VENDEDORES**, **VEICULOS** e **VENDAS**.

O backend utiliza o esquema SQL oficial da disciplina e mantém:

```
spring.jpa.hibernate.ddl-auto=none
```

O Hibernate não deve criar nem alterar tabelas do banco.

### Base URL

Ambiente local:

```
http://localhost:8080
```

Os endpoints abaixo utilizam o prefixo `/api`.

---

## 1. Clientes

Base:

```
/api/clientes
```

O cadastro de clientes respeita as especializações:

- **FISICA** → registro em `clientes` + `personas_fisicas`;
- **JURIDICA** → registro em `clientes` + `personas_juridicas`.

### POST `/api/clientes`

Cadastra um cliente.

#### Pessoa física

```
{
  "nome": "Mariana Albuquerque",
  "email": "mariana.albuquerque@example.com",
  "telefone": "81999998888",
  "rua": "Rua das Acácias",
  "numero": "145",
  "cep": "50000000",
  "tipo": "FISICA",
```

```
"cpf": "52998224725",  
"cnpj": null  
}
```

### Pessoa jurídica

```
{  
  "nome": "Autovale Comércio de Veículos Ltda",  
  "email": "contato@autovale.com.br",  
  "telefone": "8133334455",  
  "rua": "Avenida Norte",  
  "numero": "2100",  
  "cep": "52000000",  
  "tipo": "JURIDICA",  
  "cpf": null,  
  "cnpj": "12345678000190"  
}
```

### Respostas principais

- 201 Created — cliente cadastrado;
- 400 Bad Request — dados inválidos ou especialização inconsistente;
- 409 Conflict — CPF ou CNPJ já cadastrado.

### GET /api/clientes

Lista todos os clientes.

### GET /api/clientes/{idCliente}

Busca um cliente pelo identificador.

### Respostas principais

- 200 OK — cliente encontrado;
- 404 Not Found — cliente inexistente.

### PUT /api/clientes/{idCliente}

Atualiza os dados do cliente.

O `idCliente` permanece o mesmo. A operação também pode atualizar a especialização PF/PJ de forma transacional.

### DELETE /api/clientes/{idCliente}

Exclui o cliente e sua especialização.

### Respostas principais

- 204 No Content — exclusão realizada;
  - 404 Not Found — cliente inexistente;
  - 409 Conflict — existem registros vinculados que impedem a exclusão.
- 

## 2. Vendedores

Base:

/api/vendedores

### POST /api/vendedores

Cadastra um vendedor.

#### Requisição

```
{  
  "nome": "Rafael Montenegro",  
  "cpf": "52998224725"  
}
```

#### Resposta

```
{  
  "matricula": 51,  
  "nome": "Rafael Montenegro",  
  "cpf": "52998224725"  
}
```

### Respostas principais

- 201 Created — vendedor cadastrado;
- 400 Bad Request — nome/CPF inválidos;
- 409 Conflict — CPF já cadastrado.

### GET /api/vendedores

Lista os vendedores.

### GET /api/vendedores/{matricula}

Busca um vendedor pela matrícula.

#### Respostas principais

- 200 OK — vendedor encontrado;
- 404 Not Found — vendedor inexistente.

#### PUT /api/vendedores/{matricula}

Atualiza nome e CPF do vendedor.

#### DELETE /api/vendedores/{matricula}

Exclui o vendedor.

#### Respostas principais

- 204 No Content — exclusão realizada;
  - 404 Not Found — vendedor inexistente;
  - 409 Conflict — vendedor possui registros vinculados, por exemplo vendas.
- 

### 3. Veículos

Base:

/api/veiculos

O modelo segue o DDL oficial:

- `veiculos` usa `chassi` como chave primária;
- `veiculos_novos` representa veículos novos;
- `veiculos_usados` representa veículos usados;
- placa e quilometragem pertencem somente a veículos usados.

Status aceitos:

disponivel  
reservado  
vendido

#### POST /api/veiculos

Veículo novo

```
{  
  "chassi": "9BDTESTE000000001",  
  "numeroNota": null,  
  "marca": "Fiat",  
  "modelo": "Pulse Audace",  
}
```

```

    "cor": "Cinza Silverstone",
    "dataFabricacao": "2026-03-15",
    "statusDisponibilidade": "disponivel",
    "valorVeiculo": 112490.00,
    "tipo": "NOVO",
    "placa": null,
    "quilometragem": null
  }

```

### Veículo usado

```

{
  "chassi": "9BWUSAD0000000001",
  "numeroNota": null,
  "marca": "Volkswagen",
  "modelo": "T-Cross Comfortline",
  "cor": "Azul Norway",
  "dataFabricacao": "2022-08-20",
  "statusDisponibilidade": "disponivel",
  "valorVeiculo": 104900.00,
  "tipo": "USADO",
  "placa": "RZE4J82",
  "quilometragem": 38750
}

```

### Regras de especialização

Para NOVO:

- placa não deve ser informada;
- quilometragem não deve ser informada.

Para USADO:

- placa é obrigatória;
- quilometragem é obrigatória;
- placa deve ser única.

### Respostas principais

- 201 Created — veículo cadastrado;
- 400 Bad Request — dados inválidos ou combinação NOVO/USADO inconsistente;
- 409 Conflict — chassi ou placa já cadastrados.

### GET /api/veiculos

Lista todos os veículos.

### **GET /api/veiculos/{chassi}**

Busca um veículo pelo chassi.

#### **Respostas principais**

- 200 OK — veículo encontrado;
- 404 Not Found — chassi inexistente.

### **PUT /api/veiculos/{chassi}**

Atualiza o veículo sem alterar o chassi.

A operação pode alterar a especialização entre **NOVO** e **USADO**. Nesse caso, a troca é feita de forma transacional entre **veiculos\_novos** e **veiculos\_usados**.

### **DELETE /api/veiculos/{chassi}**

Exclui a especialização e o registro principal do veículo.

#### **Respostas principais**

- 204 No Content — exclusão realizada;
  - 404 Not Found — veículo inexistente;
  - 409 Conflict — existem registros vinculados que impedem a exclusão.
- 

## **4. Vendas**

Base:

/api/vendas

O modelo segue:

- **numeroNota** — chave primária gerada automaticamente;
- **idCliente** — cliente existente;
- **matriculaVendedor** — vendedor existente;
- **valorTotalVenda** — deve ser maior que zero;
- **dataDaVenda** — data da venda.

### **POST /api/vendas**

Cadastra uma venda.

### Requisição

```
{
  "idCliente": 2,
  "matriculaVendedor": 2,
  "valorTotalVenda": 127450.00,
  "dataDaVenda": "2026-08-20"
}
```

### Resposta

```
{
  "numeroNota": 51,
  "idCliente": 2,
  "matriculaVendedor": 2,
  "valorTotalVenda": 127450.00,
  "dataDaVenda": "2026-08-20"
}
```

### Respostas principais

- 201 Created — venda cadastrada;
- 400 Bad Request — valor, data ou identificadores inválidos;
- 404 Not Found — cliente ou vendedor informado não existe.

### GET /api/vendas

Lista todas as vendas.

### GET /api/vendas/{numeroNota}

Busca uma venda pelo número da nota.

### Respostas principais

- 200 OK — venda encontrada;
- 404 Not Found — venda inexistente.

### PUT /api/vendas/{numeroNota}

Atualiza cliente, vendedor, valor e data da venda, preservando `numeroNota`.

### DELETE /api/vendas/{numeroNota}

Exclui uma venda quando não existem registros dependentes.

### Respostas principais

- 204 No Content — exclusão realizada;
  - 404 Not Found — venda inexistente;
  - 409 Conflict — existem veículos ou outros registros vinculados à venda.
- 

## 5. Padrão de erros

Erros de validação de DTO retornam 400 Bad Request com os campos inválidos:

```
{  
  "valorTotalVenda": "0 valor total da venda deve ser maior que zero"  
}
```

Recurso inexistente:

```
{  
  "erro": "Venda não encontrada: 999999"  
}
```

Conflito por duplicidade:

```
{  
  "erro": "Placa já cadastrada: RZE4J82"  
}
```

Conflito de integridade referencial:

```
{  
  "erro": "Não é possível concluir a operação porque existem registros vinculados"  
}
```

---

## 6. Códigos HTTP utilizados

| Código          | Significado                                     |
|-----------------|-------------------------------------------------|
| 200 OK          | Consulta ou atualização realizada               |
| 201 Created     | Cadastro realizado                              |
| 204 No Content  | Exclusão realizada                              |
| 400 Bad Request | Dados ou regras de negócio inválidos            |
| 404 Not Found   | Recurso relacionado ou principal não encontrado |
| 409 Conflict    | Duplicidade ou integridade referencial          |

---



## 7. Observações para integração com o frontend

Os contratos JSON desta documentação correspondem aos DTOs expostos pelo backend.

No ambiente local, os endpoints CRUD estão disponíveis diretamente em:

`http://localhost:8080/api/...`

A integração em Docker pode utilizar `/api` via proxy/Nginx conforme a configuração da infraestrutura do projeto.

Os identificadores utilizados pelo frontend devem seguir o banco oficial:

- Cliente: `idCliente`;
- Vendedor: `matricula`;
- Veículo: `chassi`;
- Venda: `numeroNota`.

As especializações devem ser enviadas usando:

Cliente: `FISICA` | `JURIDICA`

Veículo: `NOVO` | `USADO`

---

## 8. Validações realizadas

Os endpoints foram validados contra o MySQL oficial da Entrega 2.

Foram verificados, entre outros cenários:

- listagem e busca individual;
- cadastro;
- atualização;
- exclusão;
- cliente PF e PJ;
- veículo novo e usado;
- troca de especialização de veículo;
- chassi duplicado;
- placa duplicada;
- cliente inexistente em venda;
- vendedor inexistente em venda;
- valores inválidos;
- respostas 400, 404 e 409;
- bloqueio de exclusão quando existem registros vinculados.